

Week11_예습과제_백재은

파이썬머신러닝완성_07_군집화

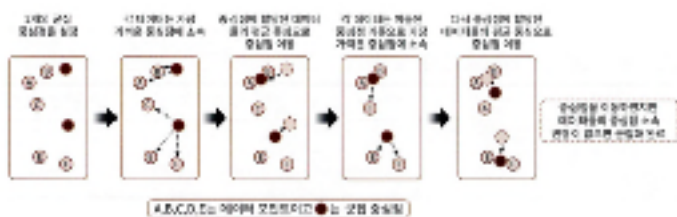
01. K-평균 알고리즘 이해

1) Intro

K-평균: 군집화(clustering)에서 가장 일반적으로 사용되는 알고리즘

군집 중심점(centroid)

-> 특정한 임의의 지점을 선택해 해당 중심에 가장 가까운 포인트들을 선택하는 군집화 기법. 선택된 포인트의 평균 지점으로 이동하고 이동된 중심점에서 다시 가까운 포인트를 선택, 다시 중심점을 평균적으로 이동하는 프로세스를 반복적으로 수행.
모든 데이터 포인트에서 더이상 중심점의 이동이 없을 경우 반복 stop, 해당 중심점에 속하는 데이터 포인트들을 군집화하는 기법



장점 -> 가장 많이 활용, 알고리즘이 쉽고 간결

단점 -> 거리 기반 알고리즘으로 속성 개수가 매우 많을 경우 정확도 down, 반복 횟수가

많을 경우 수행 시간 slow, 몇 개의 군집을 선택해야 할지 가이드 어려움

2) 사이킷런 KMeans 클래스 소개

- KMeans 클래스

초기화 파라미터

->

```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=300, tol=0.0001,
                              precompute_distances='auto', verbose=0, random_state=None,
                              copy_x=True, n_jobs=-1, algorithm='auto')
```

- 중요 파라미터:

n_clusters, 군집화할 개수, 군집 중심점의 개수 / init, 초기에 군집 중심점의 좌표를 설정할 방식 / max_iter, 최대 반복 횟수

- fit과 fit_transform 메서드 이용하여 수행

- 데이터 세트: 붓꽃의 sepal과 petal의 길이와 너비에 따른 품종 분류 -> 각 데이터의 군집화 결정, 분류 값과 비교

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

-> 데이터 로드 및 붓꽃 데이터 추출, dataframe으로 변경

-> 붓꽃 데이터 세트 3개 그룹 군집화

-> Means 객체: n_cluster = 3, 초기 중심 설정 방식 = k-means++, 최대 반복 횟수 = max_iter = 300

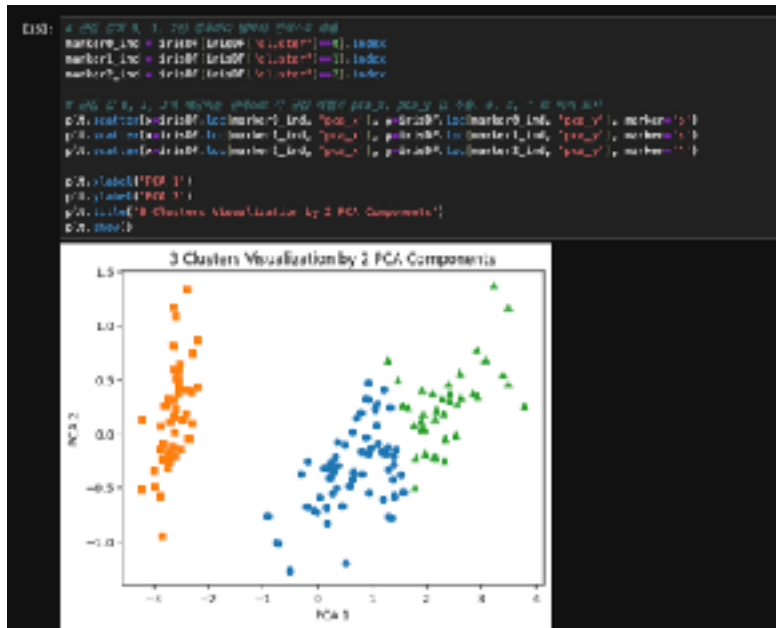
-> 실제 붓꽃 품종 분류 값과 얼마나 차이 나는가?

-> target = 붓꽃 데이터 세트의 타
겟 값 / cluster = labels_

-> group by 연산 적용하여 값 개
수 비교

-> 붓꽃 데이터 세트 군집화 시각화

-> PCA를 통해 4개 속성 2개 차원 축소, 2차원 평면상 시각화



-> pca_x = X좌표 값, pca_y = Y좌표 값

-> cluster 0 = 마커 'o', cluster 1 = 마커 's', cluster 2 = 마커 '^'

-> 산점도로 시각화

-> cluster 1에 비해 0과 2가 명확 분리 X

4) 군집화 알고리즘 테스트를 위한 데이터 생성

- 사이킷런의 데이터 생성기:

- > make_blobs(): 개별 군집의 중심점과 표준 편차 제어 기능 추가

- > make_classification(): 노이즈를 포함한 데이터

- make_blobs() 호출 파라미터

- n_samples, n_features, centers, cluster_std

```
[4]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import os

X, y = make_blobs(n_samples=200, n_features=2, centers=3, cluster_std=0.5, random_state=0)
print(X.shape, y.shape)

# y target 값 저장
cluster = y
cluster = np.array(cluster, dtype=int)
print(unique, counts)

(200, 2) (200,)
```

-> X, y =

make_blobs(n_samples=200,
n_features=2, centers=3,
random_state=0)을 호출

-> 200개의 레코드와 2개의 피처가
3개의 군집화 기반 분포도를 가진 피
쳐 데이터 세트 X와, 3개의 군집화 값
을 가진 타겟 데이터 세트 y가 반환

```
[23]: import pandas as pd

clusterDF = pd.DataFrame(data=X, columns=['ftr1', 'ftr2'])
clusterDF['target'] = y
clusterDF.head(3)
```

[23]:

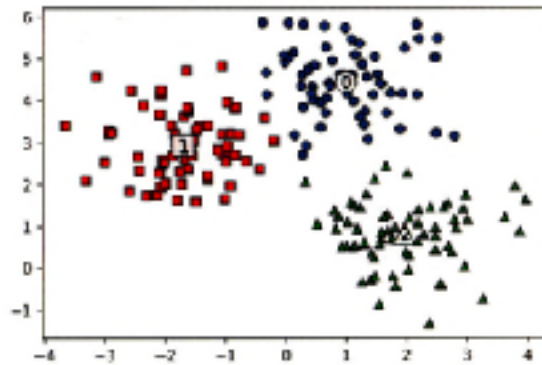
	ftr1	ftr2	target
0	-1.692427	3.622025	2
1	0.687940	4.428967	0
2	1.00228	4.606317	0

-> dataframe으로 변경



-> 데이터 군집화 분포 산점도

[Output]



-> Means 군집화 수행 후 군집별로 시각화

-> fit_predict(X) 수행, X 데이터 군집화 = 'kmeans_label' 칼럼

```
[31]: print(clusterDB.groupby('target')['kmeans_label'].value_counts())
```

target	kmeans_label	count
0	2	66
0	1	1
1	0	67
2	1	65
2	0	1

Name: count, dtype: int64

-> 매핑 확인

-> Target 00이 cluster label 0으로, target 10이 label 2로, target 2가 label 1로 거의 대부분 잘 매핑

02. 군집 평가 (Cluster Evaluation)

1) Intro

- 대부분의 군집화 데이터 세트 <=> 붓꽃 데이터 세트 ... 비교할 만한 타겟 테이블 X
- 군집화가 효율적으로 수행되었는지 확인할 수 있는 지표 = 실루엣 분석

2) 실루엣 분석의 개요

- 실루엣 분석: 각 군집 간의 거리가 얼마나 효율적으로 분리되어 있는지
- 효율적으로 잘 분리됨 = 다른 군집과 떨어져 있고 동일 군집끼리 데이터는 가까움
- 실루엣 계수: 개별 데이터가 가지는 군집화 지표

- 특정 데이터 포인트의 실루엣 계수: 해당 데이터 포인트와 같은 군집 내에 있는 다른 데이터 포인트와의 거리를 평균한 값 $a(i)$, 해당 데이터 포인트가 속하지 않은 군집 중 가장 가까운 군집과의 평균 거리 $b(i)$ 기반 계산

- 정의

$$s(i) = \frac{(b(i) - a(i))}{(\max(a(i), b(i)))}$$

- 특징: -1 ~ 1 사이의 값, 1로 가까워질수록 근처의 군집과 더 멀리 떨어져 있음 / - 값 = 아예 다른 군집에 할당됨

- 사이킷런 실루엣 분석 메서드

- `> sklearn.metrics.silhouette_samples(X, labels, metric='euclidean**kwargs)`
- `> sklearn.metrics.silhouette_score(X, labels, metric='euclidean', sample_size=None, **kwargs)`
- 좋은 군집화의 기준 조건: 실루엣 계수의 평균값이 0~1, 1에 가까울 수록 굿!! / 개별 군집의 평균값의 편차가 크지 않아야 함

3) 붓꽃 데이터 세트를 이용한 군집 평가

```
[14]: from sklearn.preprocessing import scale
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
# 군집 크기 결정할 때의 편의를 위해
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.pyplot as plt
import numpy as np

import pandas as pd
import itertools

iris = load_iris()
feature_names = ('sepal_length', 'sepal_width', 'petal_length', 'petal_width')
X = pd.DataFrame(iris.data, columns=feature_names)
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=100, random_state=0).fit(X)
iris['cluster'] = kmeans.labels_

# iris 데이터에 대한 실루엣 계수 계산
score_samples = silhouette_samples(X, iris['cluster'], metric='euclidean')
print("silhouette_samples() returns an array", score_samples.shape)

# 이 값을 실루엣 계수 평균 값
iris['silhouette_coeff'] = score_samples

# 모든 군집에 대한 실루엣 계수 평균
average_score = silhouette_score(X, iris['cluster'], metric='euclidean')
print("붓꽃 데이터에 대한 Silhouette Analysis Score [0.38].format(average_score)")

print(iris.head(2))

silhouette_score(X, iris['cluster'], metric='euclidean')
# 붓꽃 데이터에 대한 실루엣 계수 평균 값
```

```
[15]: sepal_length sepal_width petal_length petal_width cluster silhouette_coeff
0      5.1         3.5          1.4         0.2         0         0.822352
1      4.9         3.0          1.4         0.2         0         0.814068
```

```
[16]: iris.groupby('cluster')['silhouette_coeff'].mean()

[16]: cluster
0      0.422352
1      0.797064
2      0.438612
Name: silhouette_coeff, dtype: float64
```

-> 데이터 세트의 군집화 결과 실루엣 분석으로 평가

-> sklearn.metrics module's silhouette_samples() & silhouette_score() 사용

-> output 3개 row = 1번 군집 ... 0.8 가량 높은 실루엣 계수 값 but 전체 평균 0.553 ... 타 군집 낮음

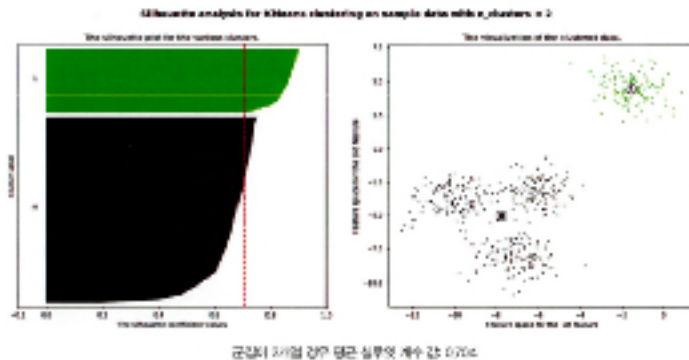
-> 군집별 평균 실루엣 계수 값

-> group by ... silhouette_coeff 컬럼의 평균값

-> 1번 0.79 에 비해 0, 2번 낮음

4) 군집별 평균 실루엣 계수의 시각화를 통한 군집 개수 최적화 방법

- 평균적으로 높은 값: 특정 군집 내의 실루엣 계수 값 높음, 타 군집은 내부 데이터끼리 거리가 멀 때 실루엣 계수 값이 낮아도… 가능!
- K-평균의 적절한 군집 개수 설정: 개별 군집별로 적당히 분리된 거리 유지, 군집 내의 데이터 뭉쳐있는 경우
- 군집의 개수 두 개



-> 1) 개별 군집에 속하는 데이터의 실루엣 계수 2차원 / X축 실루엣 계수 값, Y축 개별 군집과 이에 속하는 데이터

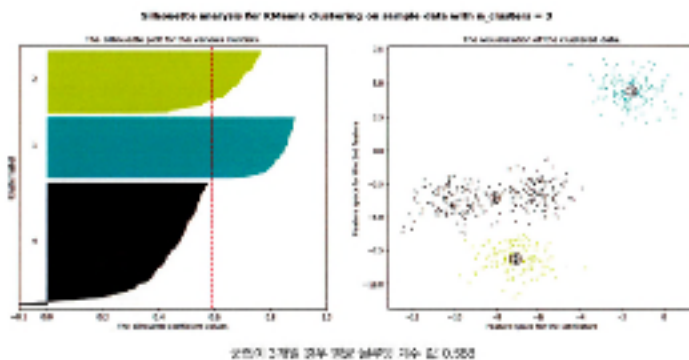
-> 1번 군집 0번 군집과 far, 내부 데이터끼리 close

-> 0번 군집 opposite

- 군집의 개수 세 개

-> 전체 데이터 평균 실루엣 계수 값 0.588

-> 1, 2번 군집 > 평균 / 0번 < 평균



- 군집의 개수 네 개

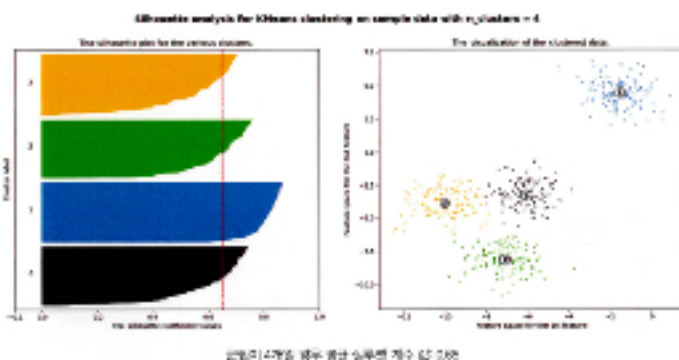
-> 개별 군집의 평균 실루엣 계수 비교적 균일

-> 1번 군집 > 평균

-> 0, 2번 절반 이상 > 평균

-> 3번 1/3 > 평균

-> 가장 이상적인 군집화 개수

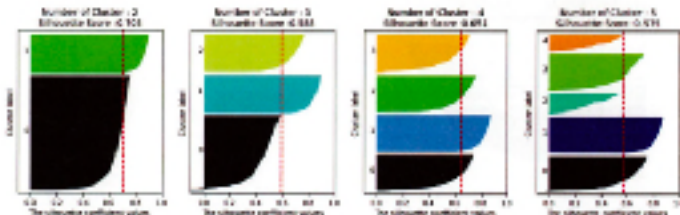


- 군집 2개, 3개, 4개, 5개일 시 군집별 평균 실루엣 계수 값 확인
- K-평균 군집화 시 2개, 3개, 4개, 5개 중 최적의 군집 개수 시각화

```
# make_blobs를 통해 군집화를 위한 4개의 군집 중심지 500개 2차원 데이터 세트 생성
from sklearn.datasets import make_blobs
# y = make_blobs(n_samples=500, n_features=2, centers=4, cluster_std=1,
#               center_box=(-10.0, 10.0), shuffle=True, random_state=1)

# 군집 개수가 2개, 3개, 4개, 5개일 때의 군집별 실루엣 계수 평균값을 시각화
visualize_silhouette([2, 3, 4, 5], X)
```

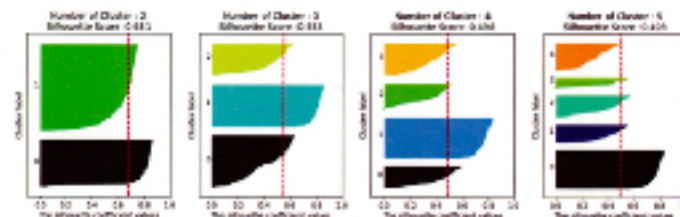
-> 4개 군집 최적



- 붓꽃 데이터 이용 K-평균 수행 시 최적 군집 개수

-> 2개 가장 최적

```
from sklearn.datasets import load_iris
iris = load_iris()
visualize_silhouette([2, 3, 4, 5], iris.data)
```



-실루엣 계수를 통한 K-평균 군집 평가

방법 = 직관, 이해 good... but 데이터 늘어나면 수행 시간 up

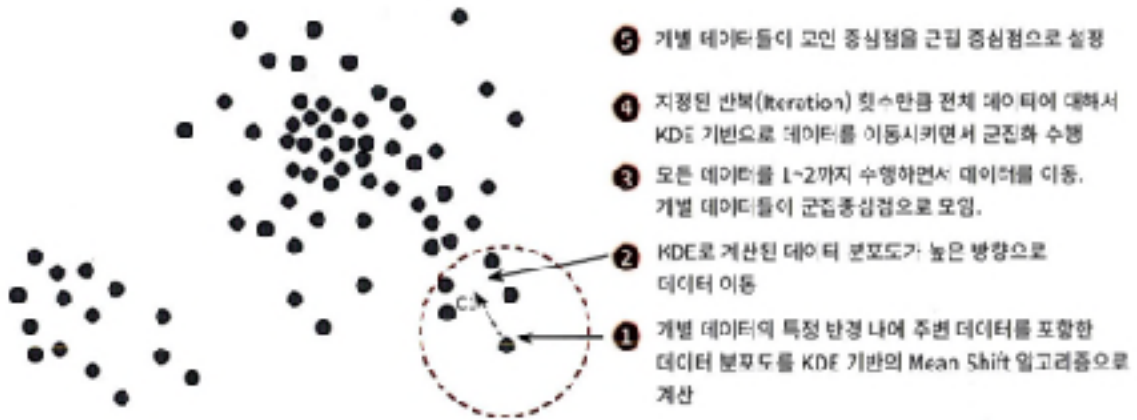
03. 평균 이동

1) 평균 이동(Mean Shift)의 개요

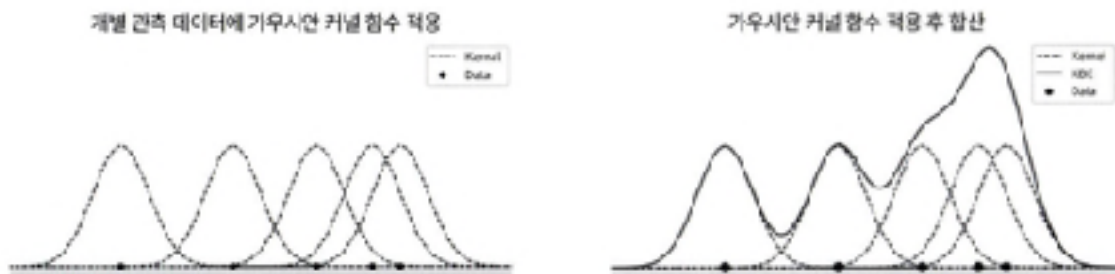
- 평균 이동: K-평균과 유사하게 중심을 군집의 중심으로 지속적으로 움직이면서 군집화 수행
- 중심을 데이터가 모여 있는 밀도가 가장 높은 곳으로 이동
- 평균 이동 군집화: 데이터의 분포도를 이용해 군집 중심점 찾기
- 군집 중심점 = 데이터 포인트가 모여있는 곳, 확률 밀도 함수 이용하여 가장 집중적으로 데이터가 모여있어 확률 밀도 함수가 피크인 점을 군집 중심점으로 선정, KDE 이용
- 평균 이동 군집화 방식: 특정 데이터를 반경 내의 데이터 분포 확률 밀도가 가장 높은 곳으로 이동하기 위해 주변 데이터와의 거리를 KDE 함수 값으로 입력, 그 반환 값을 현재 위치에서 업데이트하면서 이동하는 방식 -> 반복 적용, 데이터의 군집 중심점 찾기

- KDE: 커널 함수를 통해 어떤 변수의 확률 밀도 함수를 추정하는 대표적 방법

관측된 데이터 각각에 커널 함수를 적용한 값을 모두 더한 후 데이터 건수로 나눠 확률 밀도 함수 추정



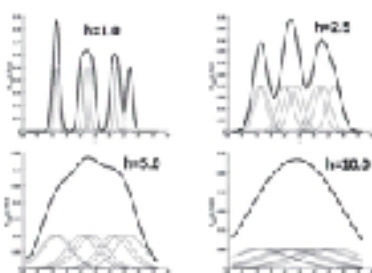
- 확률 밀도 함수 PDF: 확률 변수의 분포를 나타내는 함수, 정규분포, 감마 분포, t-분포 등...
- 대표적인 커널 함수: 가우시안 분포 함수 사용



- KDE 커널 함수식: K = 커널 함수, x = 확률 변수값, x_i = 관측값, h = 대역폭

$$KDE = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- h 값을 증가시키면서 변화되는 KDE



-> 작은 h 값 좁고 뾰족한 KDE, 변동성 큰 방식으로 확률 밀도 함수 추정, 과적합하기 쉬움

-> 큰 h 값 과도하게 평활화된 KDE로 인해 지나치게 단순화된 방식으로 확률 밀도 함수를 추정하며 결과적으로 과소적합하기 쉬움

- 사이킷런 MeanShift 클래스: 초기화 파라미터 bandwidth ... KDE 대역폭 h와 동일
- 사이킷런 estimate_bandwidth() 함수: 최적의 대역폭 계산

```
[R0]: import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import MeanShift

X, y = make_blobs(n_samples=200, n_features=2, centers=3, cluster_std=0.7, random_state=0)

meanshift = MeanShift(bandwidth=0.6)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))

cluster labels 유형: [0 1 2 3 4 5]
```

-> make_blobs() ...
cluster_std = 0.7 로 정한 3개
군집의 데이터

-> bandwidth = 0.8

-> 군집 = 0 ~ 5까지 6개 분류

- bandwidth = 1.0, MeanShift 수행

```
[R1]: from sklearn.cluster import estimate_bandwidth

bandwidth = estimate_bandwidth(X)
print('bandwidth: %f' % round(bandwidth, 3))

bandwidth: 1.610
```

-> 3개의 군집으로 군집화

- estimate_bandwidth() 파라미터로 피쳐 데이터 세트 입력, 최적화된 bandwidth 값 반환

```
[R1]: from sklearn.cluster import estimate_bandwidth

bandwidth = estimate_bandwidth(X)
print('bandwidth: %f' % round(bandwidth, 3))

bandwidth: 1.610
```

- bandwidth 를 평균 이동 입력값으로 적용, 동일한 데이터 세트에 군집화 수행

```
import pandas as pd

clusterDF = pd.DataFrame(data=X, columns=['feat1', 'feat2'])
clusterDF['target'] = y

# estimate_bandwidth()로 최적의 bandwidth 계산
best_bandwidth = estimate_bandwidth(X)

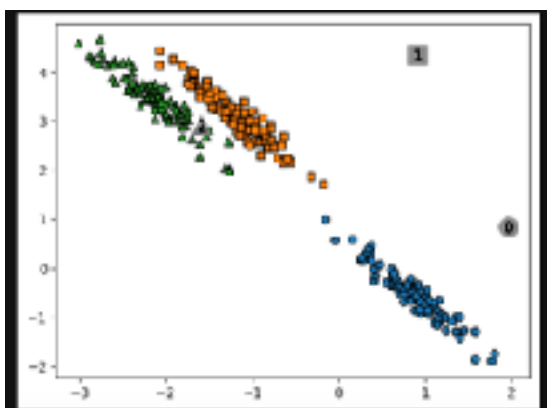
meanshift = MeanShift(bandwidth=best_bandwidth)
cluster_labels = meanshift.fit_predict(X)
print('cluster labels 유형:', np.unique(cluster_labels))
```

[Output]

cluster labels 유형: [0 1 2]

-> 3개 군집으로 구성

- 시각화



- target 값과 군집 label 값 비교

```
print(clusterOf_graphy('target')['nearshift_label'].value_counts())
```

[Output]

target	nearshift_label	
0	0	67
1	1	67
2	2	66

-> 1:1 매칭 군

04. GMM (Gaussian Mixture Model)

1) GMM 소개

- GMM 군집화: 군집화를 적용하고자 하는 데이터가 여러 개의 가우시안 분포를 가진 데이터 집합들이 섞여 생성된 것이라는 가정하에 군집화 수행하는 방식
- 가우시안 분포 (= 정규 분포): 좌우 대칭형의 종 형태 연속 확률
- 정규 분포: 평균 μ 를 중심으로 높은 데이터 분포도, 좌우 표준편차 1에 전체 데이터 68.27%, 좌우 표준편차 2에 전체 데이터 95.45% / 평균 0, 표준편차 1인 정규 분포 => 표준 정규 분포
- GMM: 여러 개의 가우시안 분포가 섞인 것으로 간주 ... 섞인 데이터 분포에 개별 유형 가우시안 분포 추출
- 세 개 정규분포 A, B, C 데이터 세트 가정 -> 합치면 전체 데이터 세트 ... 서로 다른 정규 분포 형태를 가진 여러 가지 확률 분포 곡선으로 구성 => 서로 다른 정규 분포에 기반해 군집화를 수행하는 것 => GMM 군집화 방식
- 모수 추정: 개별 정규 분포의 평균과 분산, 각 데이터가 어떤 정규 분포에 해당되는지의 확률
- 사이킷런 GaussianMixture 클래스: GMM의 EM 방식을 통한 모수 추정 군집화 지원

2) GMM을 이용한 붓꽃 데이터 세트 군집화

- 확률 기반 군집화 GMM과 거리 기반 군집화 K-평균
- GaussianMixture 객체의 가장 중요한 초기화 파라미터: n_components = 모델의 총 개수
- 3으로 설정하고 군집화 수행

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=3, random_state=0).fit(iris.data)
gmm_cluster_labels = gmm.predict(iris.data)

# 군집화 결과를 irisDF의 'gmm_cluster' 컬럼명으로 저장
irisDF['gmm_cluster'] = gmm_cluster_labels
irisDF['target'] = iris.target

# target 값에 따라 gmm_cluster 값이 어떻게 매핑됐는지 확인
irisDF_results = irisDF.groupby('target')['gmm_cluster'].value_counts()
irisDF_results.reset_index(inplace=True)
```

[Output]

target	gmm_cluster	
0	0	50
1	2	45
2	1	5

-> GaussianMixture 객체의 fit(피쳐 데이터 세트)와 predict(피쳐 데이터 세트)를 수행해 군집을 결정한 뒤 irisDF DataFrame에 'gmm_cluster' 컬럼명으로 저장하고 나서 타깃별로 군집이 어떻게 매핑됐는지 확인

-> Target 0은 cluster 0으로, Target 2는 cluster 1로 모두 잘 매핑

-> Target 1만 cluster 2로 45개(90%), cluster 1로 5개(10%) 매핑

```

kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=100, random_state=0, verbose=0)
kmeans.fit(X_train)
kmeans_cluster_labels = kmeans.predict(X_test)
iris['kmeans_cluster'] = kmeans_cluster_labels
iris_result = iris.groupby('target')['kmeans_cluster'].value_counts()
print(iris_result)

```

[Output]

target	kmeans_cluster
0	1
1	0
2	2
2	2
0	1

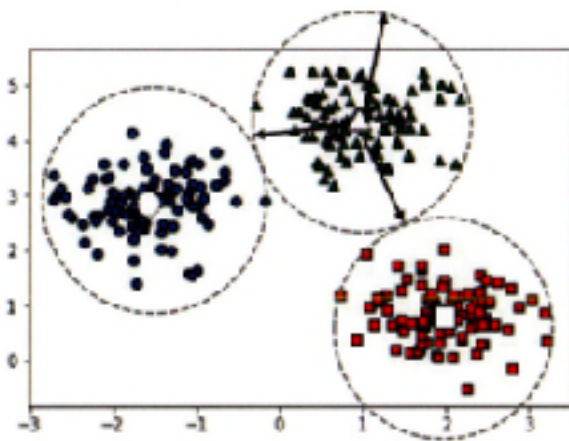
-> K-평균: 평균 거리 중심으로 중심을 이동하면서 군집화를 수행하는 방식

-> 원형으로 흩어져 있는 경우에 효과적

3) GMM과 K-평균의 비교

- KMeans: 원형의 범위에서 군집화 수행, 데이터 세트가 원형의 범위를 가질수록 군집화 효율 더욱 높아
- make_blobs() 군집의 수 3개, cluster_std를 0.5로 설정해 군집 내 데이터 뭉치게 유도하여 데이터 세트에 KMeans 적용 결과 ... cluster_std를 작게 설정하면 데이터 원형 형태로 분산

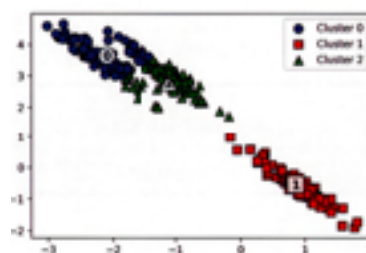
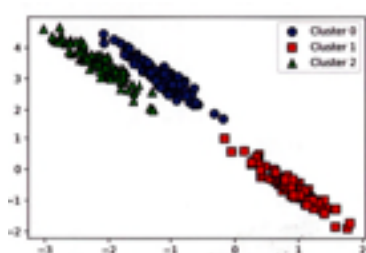
Kmeans는 원형의 범위를 가지고 Clustering을 수행



-> 개별 군집의 중심에서 원형의 범위 데이터 군집화

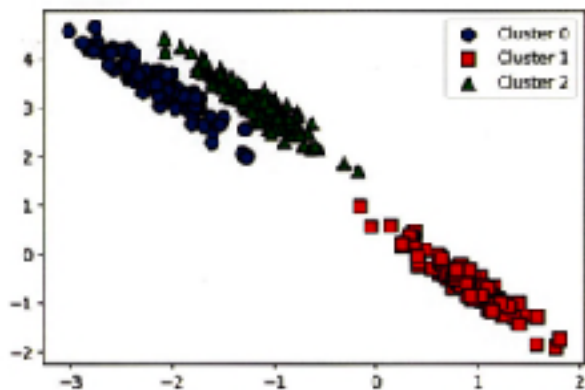
- 데이터가 원형의 범위로 퍼져 있지 않는 경우 => visualize_cluster_plot 함수 시각화

- clusterobj: 사이킷런의 군집 수행 객체. KMeans나 GaussianMixture의 fit()와 predict()로 군집화를 완료한 객체. 만약 군집화 결과 시각화가 아니고 make_blobs()로 생성한 데이터의 시각화일 경우 None 입력
- dataframe: 퍼져 데이터 세트와 label 값을 가진 DataFrame
- label_name: 군집화 결과 시각화일 경우 dataframe 내의 군집화 label 값이면, make_blobs() 결과 시각화일 경우는 dataframe 내의 target 컬럼명
- iscenter: 사이킷런 Cluster 객체가 군집 중심 좌표를 제공하면 True, 그렇지 않으면 False



-> KMeans 군집화 정확성 low

- GMM으로 군집화 수행



-> 데이터 분포 방향에 따라 정확하게 군집화

- make_blobs()의 target 값과 KMeans, GMM의 군집 Label 값을 서로 비교

```
### KMeans Clustering ###
target  kmeans_label
0      2             73
      0             27
1      1            190
2      0             86
      2             14
Name: kmeans_label, dtype: int64

### Gaussian Mixture Clustering ###
target  gmm_label
0      2          100
1      1          100
2      0          100
Name: gmm_label, dtype: int64
```

-> KMeans의 경우 군집 1 번만 정확히 매핑, 나머지 군집의 경우 target 값과 어긋나는 경우가 발생

-> GMM의 경우 군집이 target 값과 잘 매칭

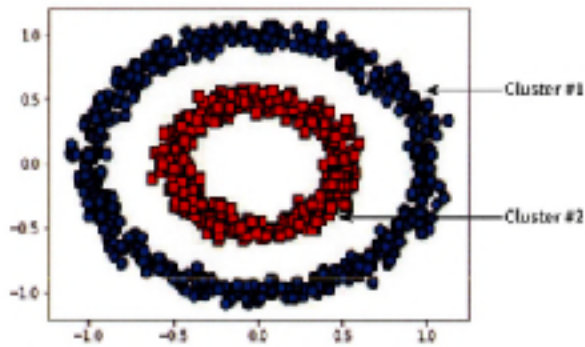
-> 단점: 수행 시간 오래 걸림

05. DBSCAN (Density Based Spatial Clustering of Applications with Noise)

1) DBSCAN 개요

- 간단하고 직관적인 알고리즘, 데이터 분포가 기하학적으로 복잡한 데이터 세트에도 효과적인 군집화 가능
- 내부의 원 모양과 외부의 원 모양 형태의 분포를 가진 데이터 세트 군집화 가정, K 평균, 평균 이동, GMM으로 효과적인 군집화 수행하기 어려움

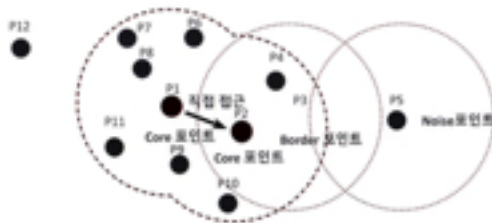
- DBSCAN의 경우 특정 공간 내에 데이터 밀도 차이를 기반 알고리즘으로 하고 있어서 복잡한 기하학적 분포도를 가진 데이터 세트에 대해서도 군집화를 잘 수행



-> 입실론 주변 영역: 개별 데이터를 중심으로 입실론 반경을 가지는 원형의 영역

-> 최소 데이터 개수: 개별 데이터의 입실론 주변 영역에 포함되는 타 데이터 개수

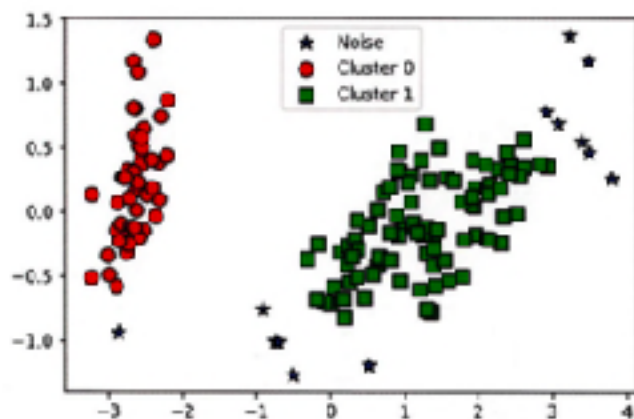
- 데이터 포인트 정의 => 핵심 포인트, 이웃 포인트, 경계 포인트, 잡음 포인트



- 사이킬릿 DBSCAN 클래스 -> DBSCAN 알고리즘 지원 => 주요 초기화 파라미터: eps, min_samples

2) DBSCAN 적용하기 - 붓꽃 데이터 세트

- DBSCAN 클래스 붓꽃 데이터 세트 군집화, eps = 0.6, min_samples = 8
- 군집화 데이터 세트를 2차원 평면에서 표현하기 위해 PCA를 이용해 2개의 피처로 압축 변환 => visualize_cluster_plot() 함수를 이용해 시각화 => irisDF의 'ftrl', 'ftr2' 칼럼에 PCA로 변환된 피처 데이터 세트를 입력

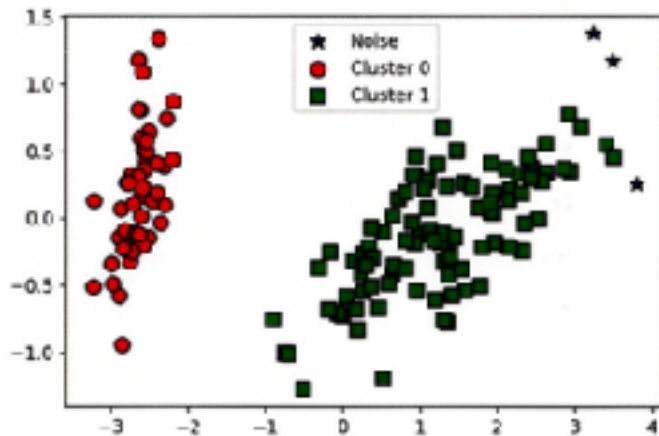


-> 별표 표현 = 노이즈

-> 특정 군집 개수로 군집 강제 X

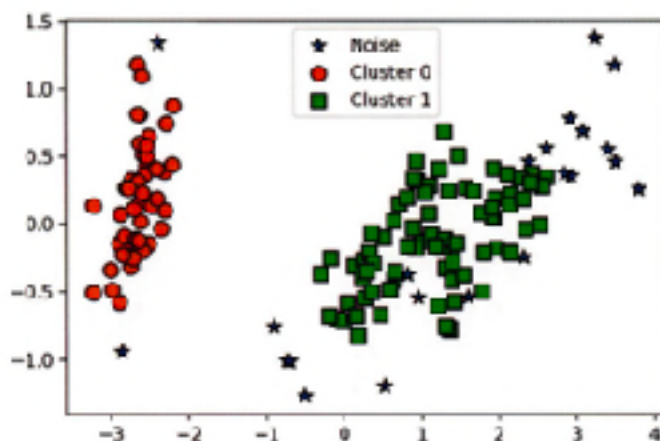
-> 일반적으로 ... eps up, 노이즈 데이터 개수 down

- eps를 기존의 0.8로 증가시키면 노이즈 데이터 수 줄어들



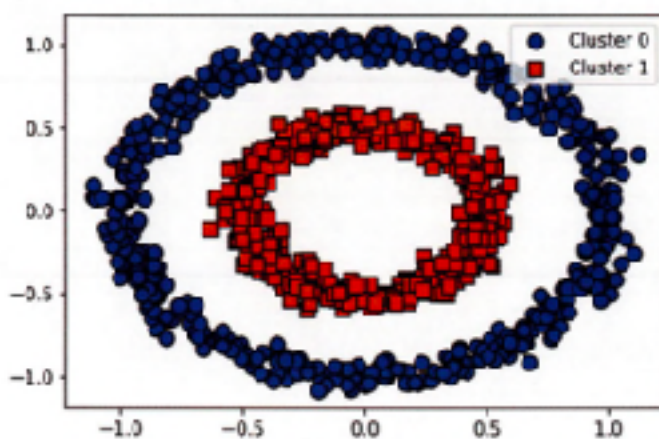
-> 노이즈 군집 -10이 3개

- eps 0.6 유지, min_samples -> 16



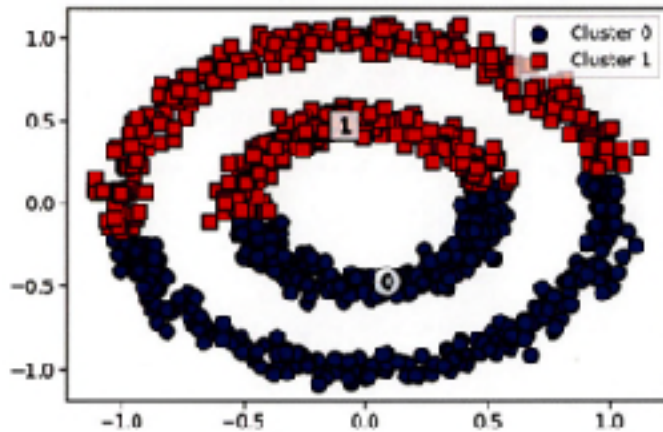
-> 노이즈 데이터가 기존보다 많이 증가

3) DBSCAN 적용하기 - make_circles() 데이터 세트

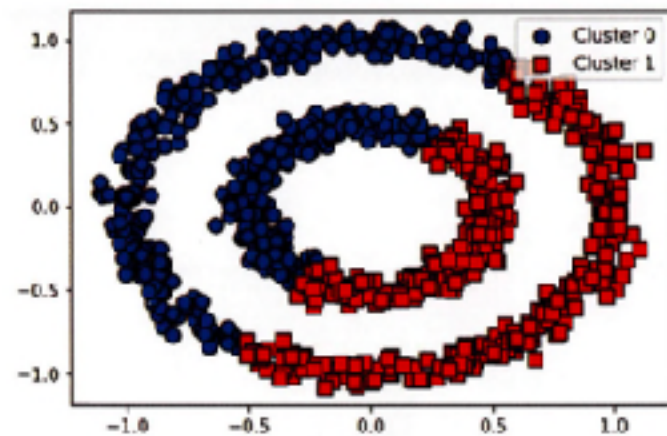


-> make_circles() 함수: 내부 원과 외부 원 형태로 돼 있는 2차원 데이터 세트

-> 파라미터 noise 노이즈 데이터 세트의 비율, factor는 외부 원과 내부 원의 scale 비율

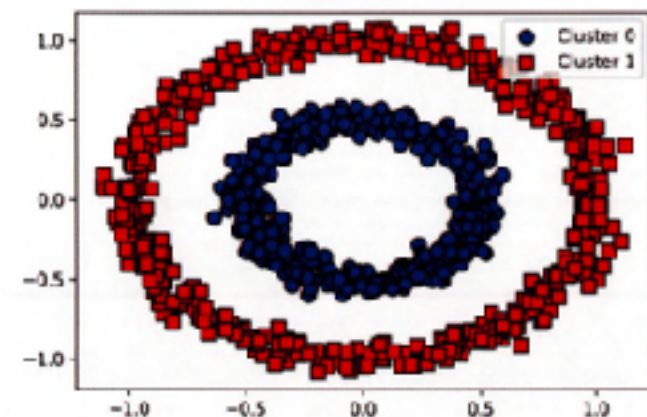


-> K-평균으로 make_circles() 데이터 세트 군집화



-> GMM) 내부와 외부의 원형으로 구성된 더 복잡한 형태의 데이터 세트에서 군집화 X

- DBSCAN 군집화 적용



-원하는 방향으로 정확히 군집화

06. 군집화 실습 - 고객 세그먼테이션

1) 고객 세그먼테이션의 정의와 기법

- 고객 세그먼테이션: 다양한 기준으로 고객을 분류하는 기법을 지칭, CRM이나 마케팅의 중요 기반 요소

- 주요 목표: 타겟 마케팅, 고객을 여러 특성에 맞게 세분화해서 그 유형에 따라 맞춤형 마케팅이나 서비스 제공하는 것
- RFM 기법: Recency, Frequency, Monetary Value

2) 데이터 세트 로딩과 데이터 클렌징

- InvoiceNo: 주문번호, 'C'로 시작하는 것은 취소 주문입니다.
- StockCode: 제품 코드(Item Code)
- Description: 제품 설명
- Quantity: 주문 제품 건수
- InvoiceDate: 주문 일자
- UnitPrice: 제품 단가
- CustomerID: 고객 번호
- Country: 국가명(주문 고객의 국적)

- 데이터 세트의 전체 건수, 칼럼 타입, Null 개수

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   InvoiceNo              541909 non-null object
1   StockCode             541909 non-null object
2   Description            540455 non-null object
3   Quantity              541909 non-null int64
4   InvoiceDate            541909 non-null datetime64[ns]
5   UnitPrice              541909 non-null float64
6   CustomerID            406829 non-null float64
7   Country                541909 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB
```

-> 사전 정제 작업 필요

- 불린 인덱싱 적용 Quantity > 0, UnitPrice > 0, CustomerID이 Not Null인 값만 다시 필터링
- > 전체 데이터 397,884로 줄어듦, Null 값 X
- Country 칼럼 -> 대다수 영국, 다른 국가 데이터 제외 => 최종 데이터 543,321 건

3) RFM 기반 데이터 가공

```
17847    7047
54096    511
52348    495
54606    290
95317    209
Name: CustomerID, dtype: int64

CustomerID
18180    70607.96
17458    70459.79
99496    70812.50
17517    70862.38
96829    67624.54
Name: total_amount, dtype: float64
```

-> 몇몇 특정 고객이 많은 주문 건수와 주문 금액

-> 판매 데이터 세트와 같이 주문번호(invoiceNo) + 상품코드 (StockCode) 레벨의 식별자

- RFM 기반의 고객 세그멘테이션 = 고객 레벨로 주문 기간, 주문 횟수, 주문 금액 데이터를 기반으로 해 세그멘테이션을 수행하는 것 -> 주문번호+상품코드 기준의 데이터를 고객 기준의 Recency, Frequency, Monetary value 데이터로 변경 -> 주문번호 기준의 데이터를 개별 고객 기준의 데이터로 Group by
- 주문번호 기준의 retail_df DataFrame에 groupby('CustomerID')를 적용해 CustomerID 기준으로 DataFrame을 새롭게 생성

```
[91]: aggregations = {
    'InvoiceDate': 'max', # 고객별 : max
    'InvoiceNo': 'count', # 고객별 : count
    'Sale_Amount': 'sum' # 고객별 : sum
}

cust_df = retail_df.groupby('CustomerID').agg(aggregations)

cust_df = cust_df.reset_index(columns=[
    'InvoiceDate': 'Recency',
    'InvoiceNo': 'Frequency',
    'Sale_Amount': 'Monetary'
])

cust_df = cust_df.reset_index()
cust_df.head(3)
```

```
[92]:
```

	CustomerID	Recency	Frequency	Monetary
0	12345.0	2011-01-18 16:01:00	1	77100.00
1	12345.0	2011-12-07 14:04:00	100	4100.01
2	12345.0	2011-12-09 12:10:00	4000	33719.73

-> DF에 groupby 호출, 반환 DFgroupby 객체에 agg() 이용

-> 인자로 대상 칼럼들과 aggregation 함수명들을 딕셔너리 형태로 입력하면 칼럼 여러 개의 서로 다른 aggregation 연산을 쉽게 수행

-> Recency 가공 x2

-> "CustomerID"로 groupby()해서 InvoiceDate' 칼럼의 max()로 고객별 가장 최근 주문 일자를 먼저 구한 뒤 추후에 가공 작업을 별도로 수행

- 2011/12/10을 현재 날짜로 간주, 가장 최근의 주문 일자를 뺀 데이터에서 일자 데이터만 추출 생성

```
import datetime as dt

cust_df['Recency'] = dt.datetime(2011, 12, 10) - cust_df['InvoiceDate']
cust_df['Recency'] = cust_df['Recency'].apply(lambda x: x.days)
print('cust_df 데이터 총행 개수: ', cust_df.shape)
cust_df.head(3)
```

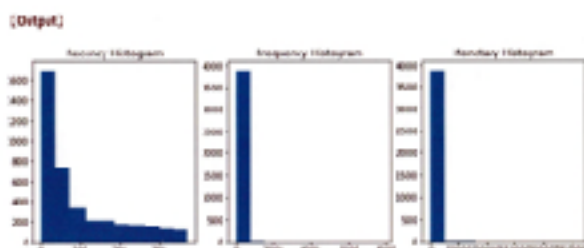
```
[Output]
```

cust_df 데이터와 칼럼 인자는 (3x3), >>

	CustomerID	Recency	Frequency	Monetary
0	12345	200	1	77100.00
1	12345	5	100	4100.01
2	12345	1	4000	33719.73

4) RFM 기반 고객 세그멘테이션

- 온라인 판매 데이터 세트 -> 소매업체의 대규모 주문 포함, 주문 횟수와 주문 금액에서 개인 고객 주문과 큰 차이 -> 매우 왜곡된 데이터 분포, 군집화가 한쪽 군집에만 집중되는 현상 발생
- 온라인 판매 데이터 세트 칼럼별 히스토그램 확인



-> 왜곡된 데이터 값 분포도

- 데이터 값 백분위 분포 확인

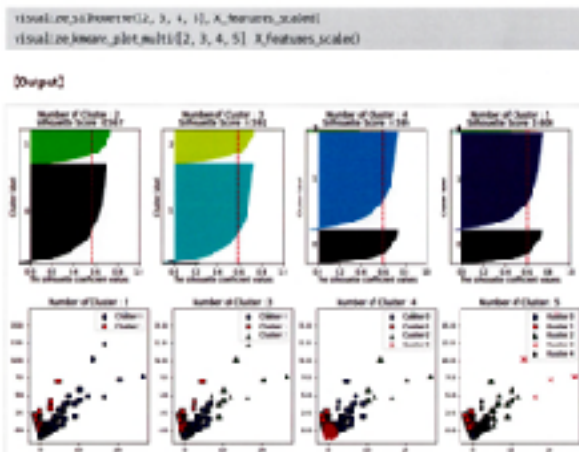
	Recency	Frequency	Monetary
count	3020.000000	3020.000000	3020.000000
mean	92.742092	90.388010	1854.385601
std	99.533485	217.000395	7482.817477
min	1.000000	1.000000	3.750000
25%	18.000000	17.000000	300.280000
50%	51.000000	41.000000	852.280000
75%	143.000000	99.250000	1576.585000
max	374.000000	7847.000000	259657.300000

-> Recency 평균 92.7 > 51

-> max 값 374 > 143

-> Frequency 평균 0.3 ... max 상위 큰 값으로 인해 왜곡 정도 높음

- K-평균 군집 적용, 중심 개수 up, 변별력 down
- StandardScaler 로 평균과 표준편차 재조정 후 K-평균 수행 ... 실루엣 스코어 0.592
- 각 군집별 실루엣 계수값



-> 군집 2개: 0번, 1번 군집이 너무 개괄적으로 군집화

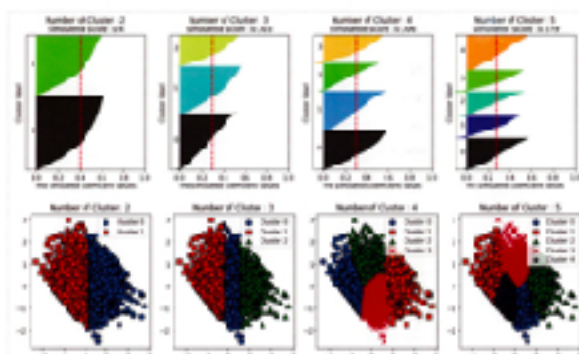
-> 군집 3개 이상: 데이터 세트 개수가 너무 작은 군집

-> 군집 3개: 0번 군집 데이터 건수 작음

-> 4개: 2번, 3번 군집 / 5개: 2, 3, 4번 군집에 속한 데이터 세트의 개수 적고 광범위

=> 거리 기반 광범위하게 분포... 의미 X

- 로그 변환 -> K-평균 알고리즘 적용, 결과 비교 ... 실루엣 스코어 0.303
- 실루엣 계수와 군집화 구성 시각화



-> 실루엣 스코어는 로그 변환하기 전보다 떨어지지만 앞의 경우보다 더 균일하게 군집화 구성

07. 정리

K-평균 (K-Means)

- 기본 원리: 거리 기반으로 군집 중심점을 이동시키며 군집화를 수행하는 알고리즘
- 특징:
 - 군집의 중심을 반복적으로 조정하여 데이터 포인트를 최적의 군집으로 분류
 - 매우 쉽고 직관적인 알고리즘
- 장점:
 - 계산 속도가 빠르고 구현이 간단함
 - 대규모 데이터에 적용 가능
- 단점:
 - 복잡한 구조의 데이터에는 한계가 있음
 - 군집의 개수(K) 설정이 필요하며 최적화가 어렵다
- 평가 방법: 실루엣 계수를 사용하여 군집의 품질을 평가

평균 이동 (Mean Shift)

- 기본 원리: 데이터가 밀집된 방향으로 군집 중심점을 이동하며 군집화 수행
- 특징:
 - 데이터의 밀도가 가장 높은 방향으로 이동하며 군집을 형성
 - 밀도 피크(모드)를 중심으로 데이터 그룹을 형성
- 장점:
 - K-평균과 달리 군집의 개수를 사전에 지정할 필요가 없음
 - 비정형 데이터, 특히 이미지나 영상 데이터의 객체 분할에 유리
- 단점:
 - 계산 비용이 높고, 대규모 데이터에는 부적합
 - 이상치(outlier)에 민감할 수 있음

가우시안 혼합 모델 (Gaussian Mixture Model, GMM)

- 기본 원리: 여러 개의 가우시안 분포를 섞어서 데이터의 밀도를 모델링하여 군집화

- 특징:

- 각 군집이 가우시안 분포로 모델링된다고 가정
- 각 데이터 포인트가 여러 군집에 속할 확률을 계산

- 장점:

- 복잡하고 다양한 분포 형태의 데이터에도 유연하게 적용 가능
- 군집의 형태가 원형이 아니어도 잘 작동

- 단점:

- 수행 시간이 오래 걸릴 수 있음
- 파라미터 추정이 어려울 수 있음

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- 기본 원리: 밀도 기반으로 군집을 형성하며, 밀도가 낮은 영역을 잡음(noise)으로 구분

- 특징:

- 핵심 포인트(core point), 이웃 포인트(border point), 경계 포인트, 잡음 포인트로 데이터 포인트를 분류
- 직접 접근이 가능한 핵심 포인트를 서로 연결하여 군집을 형성

- 장점:

- 복잡한 형태의 데이터와 이상치가 포함된 데이터에도 강력한 성능을 보임
- 군집의 개수를 사전에 설정할 필요가 없음

- 단점:

- 밀도 매개변수 설정이 까다로울 수 있음
- 데이터의 분포가 매우 균일하지 않으면 성능이 저하될 수 있음

-